

Sri Lanka Institute of Information Technology



IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 05

Student ID	IT24103883
Student Name	Jayasinghe B V A D K
Campus/ Center name	Malabe
Specialization	Data Science
Batch No	2.2

Activity 1

Q1

```
$ nano activity1.c
$ nano activity1.c
$ gcc -oactivity1 activity1.c
$ ./activity1
Hello from the thread!
Thread has finished execution.
```

Q2

print_message

printf

perror

pthread_create

pthread_join

pthread_exit

Q3

1. It begins in the main thread, which declares a thread identifier (thread_id) and attempts to spawn a new "worker" thread using pthread_create.
2. If successful, the new thread runs the print_message function independently, while the main thread uses pthread_join to synchronize, ensuring it doesn't finish until the worker thread is done.
3. The program demonstrates how to create, execute, and wait for a single thread to complete its task before the entire process terminates.

Activity 2

```
GNU nano 7.2 activity1.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 3

void* print_thread_num(void*arg){
    int thread_num=*(int*)arg;
    printf("Hello from the thread number:%d\n",thread_num);
    pthread_exit(NULL);
}

int main(){
    pthread_t threads[NUM_THREADS];
    int thread_args[NUM_THREADS];
    int i;

    for(i=0;i<NUM_THREADS;i++){
        thread_args[i]=i+1;
        printf("Main: creating thread %d\n",thread_args[i]);

        if (pthread_create(&threads[i],NULL,print_thread_num,&thread_args[i])!=0){
            perror("Thread creation failed");
            return 1;
        }
    }

    for(i=0;i<NUM_THREADS;i++){
        pthread_join(threads[i],NULL);
    }
}
```

output

```
$ nano activity1.c
$ gcc -oactivity1 activity1.c
$ ./activity1
Main: creating thread 1
Main: creating thread 2
Main: creating thread 3
Hello from the thread number:2
Hello from the thread number:1
Hello from the thread number:3
Main:All threads have finished.
$
```

Activity 3

output

```
$ nano activity3.c
$ gcc -oactivity3 activity3.c
$ ./activity3
sum =30
$
```

Q1

It defines a new data type called Numbers that groups related data into a single unit

Q2

To maintain compatibility with the pthread_create function prototype, which requires a generic pointer to accept any data type.

Q3

This line performs a type cast to convert the generic void* pointer back into a specific Numbers* pointer so the data can be accessed.

Activity 4

```
GNU nano 7.2 Activity4.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

typedef struct {
    int a;
    int b;
} Numbers;

typedef struct {
    int arr[4];
    int size;
} ArrayData;

void* calculate_sum (void* arg) {
    Numbers* nums = (Numbers*)arg;
    int sum = nums->a + nums->b;
    printf("Sum = %d\n", sum);
    return NULL;
}

void* calculate_product(void* arg){
    Numbers* nums = (Numbers*)arg;
    int product = nums->a * nums->b;
    printf("Product = %d\n", product);
    return NULL;
}
```

```

void* calculate_array_sum(void* arg) {
    ArrayData* data = (ArrayData*)arg;
    int total = 0;
    for (int i=0; i<data->size; i++) {
        total += data->arr[i];
    }
    printf("Array Total Sum = %d\n",total);
    return NULL;
}

int main() {
    pthread_t t1, t2, t3;

    Numbers n;
    printf("Enter two numbers: ");
    scanf("%d %d", &n.a, &n.b);

    ArrayData data;
    data.size = 4;
    printf("Enter 4 numbers for the array: ");
    for (int i=0; i<4; i++) {
        scanf("%d", &data.arr[i]);
    }
}

```

```

pthread_create(&t1, NULL, calculate_sum, &n);
pthread_create(&t2, NULL, calculate_product,&n);
pthread_create(&t3, NULL, calculate_array_sum, &data);

pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_join(t3, NULL);

return 0;
}

```

```

$ nano Activity4.c
$ gcc -oactivity Activity4.c
$ ./activity
Enter two numbers: 6 7
Enter 4 numbers for the array: 5 6 7 8
Sum = 13
Product = 42
Array Total Sum = 26

```